

不平衡資料建模

數據集中各個類別的樣本比例存在顯著差異

不平衡數據問題概述

- 多數類別 (Majority Class)
 - 樣本數量占絕大部分的類別。
- 少數類別 (Minority Class)
 - 樣本數量極少，通常也是研究中更感興趣或更重要的類別（如病患、詐騙行為）

應用領域

- **醫療診斷**：罕見疾病的患者比例遠低於健康人群
- **金融欺詐檢測**：信用卡盜刷或非法轉帳樣本在海量正常交易中極為稀少
- **資訊安全**：網路入侵檢測、惡意軟體識別
- **工業檢測**：設備故障預測、缺陷檢測
- **電子商務**：客戶流失預測、直效營銷回應預測

建立模型技術困難

- **性能偏見 (Performance Bias)**：大多數學習演算法假設類別分布平衡，並以「最大準確率」為目標。導致模型會向多數類別傾斜，雖然整體準確率很高，但對少數類別的預測能力差。
- **小分取 (Small Disjuncts)**：少數類別常被分解成多個小的子集群，這些小集群容易與雜訊混淆，增加學習複雜度。
- **數據重疊 (Overlapping)**：少數類與多數類在特徵空間中可能存在重疊，導致決策邊界難以劃分。
- **數據漂移 (Dataset Shift)**：訓練集與測試集的分布可能不一致，這在不平衡場景下會導致性能大幅下降。
- **資訊丟失或過擬合**：在處理數據時，刪除多數類樣本可能丟失重要資訊（欠取樣），而重複少數類樣本則可能導致模型過擬合（過取樣）。

模型評估指標謬誤

- 不平衡數據中，準確率（Accuracy）是一個具有誤導性的指標
 - 若 99% 的樣本為正常，1% 為異常。一個簡單地將所有樣本預測為「正常」的模型，其準確率高達 99%，但它對異常樣本的偵測率為 0%，完全失去了實際應用價值
- 替代指標

指 標 名 稱	意 義	說 明
精確率	在預測為正類的樣本中，真正為正類的比例	
召回率	在所有實際為正類的樣本中，被正確預測出來的比例	
F1-Score	精確率與召回率的調和平均數，平衡兩者的表現	
G-mean	多數類與少數類準確率的幾何平均值，衡量兩類性能的均衡性	
AUC-ROC	受試者工作特徵曲線下的面積，對類別分布不敏感，適合衡量整體區分能力	

可行解決方案

- 預處理方法

- 隨機過取樣 (Random Over-sampling)：通過複製少數類樣本增加其數量。缺點是容易導致過擬合。
- 隨機欠取樣 (Random Under-sampling)：通過刪除多數類樣本減少其數量。缺點是可能丟失重要資訊。
- SMOTE (Synthetic Minority Over-sampling Technique)：通過插值法產生「合成」的少數類樣本，而非單純複製，有助於擴展決策邊界並降低過擬合。
- ADASYN (Adaptive Synthetic Sampling)：根據樣本的學習難度分配權重，為難以學習的少數類樣本生成更多合成數據。
- 數據清洗（如 Tomek Links）：刪除多數類中的雜訊或邊界模糊樣本，使類別邊界更清晰。

- **演算法處理**

- 代價敏感學習 (Cost-Sensitive Learning)：為不同的分類錯誤分配不同的代價。例如，漏報一個癌症病例的代價（罰分）遠高於誤報。
- 權重調整 (Class Weighting)：在訓練過程中，賦予少數類樣本更高的權重，迫使模型關注這些樣本。
- 門檻調整 (Threshold Moving)：不再使用固定的 0.5 作為分類閾值，而是根據實際類別分布或成本函數移動閾值。
- 單類學習 (One-class Learning)：當數據極端不平衡時，將問題視為異常檢測，僅學習正常類別的分布（如一類支持向量機 One-class SVM）

- 集成與混合方法

- Bagging (套袋法) : 如隨機森林。在不平衡數據中，可以使用 Balanced Random Forest，在每個子樹訓練時進行平衡取樣。([balanced bagging classifier](#))
- Boosting (提升法) :
 - RUSBoost : 將欠取樣與 AdaBoost 結合，每輪迭代都進行平衡。
 - SMOTEBoost : 將 SMOTE 合成技術與 Boosting 結合。
- 混合取樣 : 同時結合過取樣 (處理少數類) 與欠取樣 (處理多數類)，以獲得最佳比例

實作

調整類別權重(class weight)及懲罰成本(penalty cost)

- 在 RandomForestClassifier 或 LogisticRegression 等模型直接設定參數
 - **class_weight='balanced'** : 根據類別頻率自動反向調整權重,。權重會與各類別在數據中出現的頻率成反比, 以補償數據分佈的不均
- 調整決策閾值 (Threshold Adjustment)
 - 傳統分類預設閾值為 0.5。在成本敏感學習中, 可以根據誤判成本的大小調低判定為正向類的門檻, 以捕獲更多重要的少數類樣本
- 決策樹剪枝調整
 - 在決策樹中, 調整葉節點的機率估計或增加剪枝時對過擬合多數類的影響, 以補償代價的不對稱

Undersampling與Oversampling實作

- 欠抽樣 (Undersampling) 與過抽樣 (Oversampling) 是處理不平衡數據最直觀的數據預處理方式，主要目的在於重新調整類別分佈，以獲得較平衡的訓練集
- 欠抽樣 (Undersampling) 的實作
 - 隨機從多數類中抽取與少數類數量相等的樣本，並忽略其餘多數類樣本
 - Scikit-learn / Imbalanced-learn 實作方式
 - RandomUnderSampler 模組，`fit_resample(X, y)`
 - 可能會損失重要資訊，因為被刪除的樣本中可能包含對模型訓練有價值的特徵
- 過抽樣 (Oversampling) 的實作
 - 隨機複製現有的少數類樣本，直到其數量與多數類持平
 - 啟用 `replace=True` (取後放回) 參數進行重複抽樣
 - 使用 RandomOverSampler 進行簡單複製
 - 使用SMOTE (合成少數類過取樣技術)
 - 容易導致過擬合 (Overfitting)，因為模型可能會過度學習重複樣本的細節而非泛化特徵

Sklearn實作

```
from imblearn.over_sampling import RandomOverSampler
```

```
oversample = RandomOverSampler(sampling_strategy='minority' )
```

- 從少數類別重新抽樣產生與多數類別同樣數量

```
oversample = RandomOverSampler(sampling_strategy=0.5)
```

- 從少數類別重新抽樣產生與多數類別1/2數量

```
from imblearn.under_sampling import RandomUnderSampler
```

```
undersample = RandomUnderSampler(sampling_strategy='majority' )
```

- 從多數類別重新抽樣產生與少數類別同樣數量

SMOTE (Synthetic Minority Over-sampling Technique)

- 「合成少數類過取樣技術」

- 以插值法 (Interpolation)，在現有的少數類樣本之間創造出全新的「合成」樣本，讓少數類的決策邊界進一步擴展到多數類的空間中，使模型學會更具泛化性的特徵，而非只是記住特定的樣本點

- 運作概念

- 尋找鄰居：針對每一個少數類樣本，計算其在同一類別中的 k 個最近鄰居（在 SMOTE 中預設 $k=5$ ）。
- 隨機選擇：根據設定的過取樣比例，從這 k 個鄰居中隨機挑選出幾個。
- 線性插值：在原始樣本與所選鄰居之間的連線上，隨機選擇一個點作為新的合成樣本。

- 實作步驟

- 導入 `imblearn.over_sampling` 中的 SMOTE 模組,。
- 建立 SMOTE 物件，通常會設定 `random_state` 以確保結果可重現,。
- 使用 `.fit_resample(X, y)` 方法，傳入原始特徵矩陣 X 與標籤向量 y ，回傳平衡後的數據,。

Sklearn實作

```
from imblearn.over_sampling import SMOTE
```

```
from collections import Counter
```

```
# 假設 x 為特徵資料，y 為目標標籤
```

```
print(f'原始數據分佈: {Counter(y)}')
```

```
# 初始化 SMOTE
```

```
smote = SMOTE(random_state=42)
```

```
# 進行合成取樣
```

```
X_resampled, y_resampled = smote.fit_resample(X, y)
```

```
print(f'重取樣後數據分佈: {Counter(y_resampled)}')
```

Multiclass Classification

多類別分類

多類別分類 (Multiclass Classification)

- 在多類別分類任務中，系統從多個選項中（超過二個）選出一個最正確的標籤
- 二元分類 (Binary Classification) 僅處理**兩個類別**（如：是或否、垃圾郵件或非垃圾郵件），而多類別分類則處理**三個或更多類別**
- **隨機森林** (Random Forest)、**XGBoost**、**決策樹** (Decision Tree)、**單純貝氏** (Naive Bayes)能支援多個類別分類
- **邏輯斯回歸** (Logistic Regression)、**感知器** (Perceptron)、**支持向量機** (SVM)本質上是為二元分類設計的，無法直接處理多類別任務
 - **一對全 (One-vs-Rest, OvR)**：為每個類別建立一個模型，將該類別與其餘所有類別進行對比
 - **一對一 (One-vs-One, OvO)**：為每一對類別組合建立一個模型，最後透過投票決定結果

多類別分類技術的元策略(meta-strategies)

- 一對全策略 (One-vs-Rest, OvR)

- 有紅、藍、綠三色，會拆解為「紅 vs. 非紅」、「藍 vs. 非藍」及「綠 vs. 非綠」三組實驗
- 系統會訓練 N 個模型，最終預測會選擇**信心程度最高（機率最大）**的那個類別
- 模型數量較少（僅 N 個），對於大型資料集或運算較慢的模型（如神經網路）更具優勢

- 一對一策略 (One-vs-One, OvO)

- 模型數量會隨著類別增加而劇增，公式為 $N*(N-1)/2$ 。例如 4 個類別就需要 6 個二元模型
- 最終結果是透過所有模型的**多數投票（Majority Vote）**來決定，或是取得分總和最高的類別
- 雖然模型數量多，但每個模型只處理部分類別的資料。在某些算法（如 SVM）中，針對子集進行訓練反而能提升性能

- scikit-learn提供 **OneVsRestClassifier** 和 **OneVsOneClassifier**，將任何二元分類器轉換為多類別分類器，建議透過交叉驗證（Cross-validation）同時測試 OvR 與 OvO，以找出最適合特定資料集的方案

Final think

- **Training Data**

- 用來學習特徵、調整權重 (Weights) 的基礎資料
- 調整模型的**超參數** (Hyperparameters)

- **Test Data**

- 模型架構、超參數全部定案，甚至已經用全部資料重新訓練好最終模型後，才會拿 Test Data 來做最後的效能評估
- 真正反映出模型在面對未來現實世界 (未見過的新資料) 時的泛化能力 (Generalization Ability)

- 切分資料：資料先切出 20% 作為 Test Data
- 交叉驗證：剩下的 80% 資料進行 K-Fold Cross-Validation。這 80% 的資料會被輪流當作訓練和驗證集，用來挑選出最棒的模型結構與參數
- 最終訓練：選定最佳的參數後，使用 80% 訓練資料重新訓練出一個最終模型（ Final Model ）
- 拿這個最終模型去預測那組 20% 的 Test Data，得到的準確率（ Accuracy、F1-score 等 ），才是該模型真正的實力。